

Safe Change Workflow

A practical guide for loosely coupled, removable, and verifiable changes to the Web AR prototype.

Development Guidelines

All future changes must keep the code loosely coupled. Prefer small modules with explicit dependencies over shared global state or tightly connected functions. When adding or changing behavior, preserve existing functionality, reduce interdependency where practical, and verify the change before stacking additional work on top.

For larger changes, refactor in small phases and commit after each verified phase so regressions are easier to isolate.

Safe Change Workflow

Treat every requested change as a small product decision plus a small architecture change. Every feature should have an owner module, explicit dependencies, a removal path, and a verification step.

Change Request

When requesting changes, include the desired behavior, what must stay unchanged, and whether any older behavior is now obsolete.

I want to change/add/remove [feature].

Desired behavior:

- ...

Keep unchanged:

- ...

Obsolete behavior to remove:

- ...

Safety requirement:

- Keep this loosely coupled.
- Do not add hidden dependencies or shared global state.
- Remove dead code if this replaces an older feature.
- Verify the app still loads and the relevant flow still works.

Implementation Rules

- Inspect the affected modules before editing.
- Keep feature logic in the smallest appropriate owner module.
- Keep `src/app.js` as the composition root only.
- Pass dependencies explicitly instead of importing from `src/app.js`.
- Avoid shared global state unless it is already part of the app state model.
- Preserve existing behavior unless the request explicitly replaces it.
- Do not keep obsolete fallback branches, unused functions, stale comments, unused imports, or dead UI code.

Obsolete Code Removal

When replacing behavior, remove the old path completely:

- unused functions
- unused state fields
- unused imports
- unused event listeners

- unused UI elements or CSS classes
- unused debug hooks
- stale comments or documentation

Search for the old feature name before committing so stale references do not remain.

Verification

After each meaningful change:

- run JavaScript syntax checks
- confirm relative module imports resolve
- start the local preview server
- smoke-test the affected UI, AR, debug, or reset flow
- review git diff for unrelated changes

Prompt Templates

Adding a feature:

Add [feature].

Behavior:

- ...

Keep unchanged:

- ...

Architecture:

- Keep it loosely coupled.
- Put the feature logic in the smallest appropriate owner module.
- Use explicit dependency injection instead of importing from app.js.
- Do not add global state unless there is no safer option.

Verification:

- Run syntax checks.
- Confirm imports resolve.
- Run the local server smoke test.
- Tell me what changed and what files own the behavior.

Changing an existing feature:

Change [existing feature] so that [new behavior].

Important:

- Preserve [specific behavior].
- Replace [old behavior] if it conflicts.
- Remove obsolete code instead of leaving fallback branches behind.
- Search for stale references after the change.

Keep the code loosely coupled and commit the change separately.

Removing a feature:

Remove [feature] completely.

Please remove:

- behavior logic
- unused state fields
- unused UI elements/classes
- unused imports
- stale debug hooks
- stale docs/comments

Do not remove unrelated code. Verify that the app still loads and that nearby features still work.

Fixing a problem:

Fix [problem or error].

Observed behavior:

- ...

Expected behavior:

- ...

How to reproduce:

1. ...
2. ...

Safety requirements:

- Identify the root cause before editing.
- Keep the fix in the smallest appropriate owner module.
- Preserve unrelated behavior and public interfaces.
- Use explicit dependencies; do not add hidden coupling or global state.
- Remove any obsolete workaround, dead branch, or stale comment replaced by the fix.
- Add or update a focused regression check when practical.

Verification:

- Reproduce the problem before the fix when possible.
- Confirm the reproduction no longer fails after the fix.
- Run syntax and import checks.
- Smoke-test the affected flow and nearby behavior.
- Report the root cause, changed files, and verification results.

Refactoring only:

Refactor [area/module] without changing behavior.

Rules:

- No feature changes.
- Keep public behavior and messages the same.
- Move code into smaller modules with explicit dependencies.
- Remove duplicated or dead code only when it is clearly unused.
- Commit after verification.

For any change that replaces old behavior, include this line:

Also remove anything made obsolete by this change.